Chordal Graph Sampling-Based Mini-batch Training Algorithm for Large Graphs

Ziyang Su[0009−0009−3947−4506], Wentao He, and Jiayuan Liu[0009−0006−4493−3250]

Shanghai Jiao Tong University, Shanghai, China
{suziyang,dlpt13}@sjtu.edu.cn, 651646673@qq.com

Abstract. Graph Neural Networks (GNNs) are powerful models for learning representations of attributed graphs. To scale GNNs to large graphs, many methods use various techniques, such as sampling and decoupling, to alleviate the "neighbor explosion" problem during mini-batch training. However, these sampling-based mini-batch training methods often suffer from greater information loss than decoupling-based methods or full-batch GNNs. Besides, most original segmentation methods for large graphs usually lose a large number of edges, resulting in suboptimal performance when performing mini-batch training. Therefore, we propose a **C**hordal **G**raph **S**ampling-based mini-batch **T**raining algorithm for GNNs on large scale graph datasets, called **CGST**. CGST includes a balanced chordal graph partition module and a batch random aggregation module to improve performance on node classification tasks while maintaining main information of the original graph structure. Experiments on three large-scale graph datasets prove the effectiveness of CGST.

Keywords: Graph neural network · Mini-batch training

1 Introduction

The Graph Neural Network(GNN) has garnered significant attention in the realm of graph-based applications, encompassing tasks such as semi-supervised node classification [10], link prediction [16], and recommender systems [14]. Within the framework of a given graph, GNN employs graph convolutional operations to compute node embeddings across multiple layers iteratively. At each layer, a node's embedding is derived by aggregating information from its neighboring nodes, subsequently undergoing one or more layers of linear transformations and nonlinear activations. The resultant embedding at the final layer is then utilized for various downstream tasks. For instance, in scenarios involving node classification, the final layer embedding is fed into a classifier to infer node labels, facilitating end-to-end training of GNN parameters.

Despite the notable achievements of GNNs in numerous graph-related applications, the training of GNNs for extensive graphs poses a significant challenge. Unlike text or images with usually constrained lengths or sizes, practical graph

data can frequently present itself in immensely expansive scales or dimensions, reflecting the complexity and magnitude inherent in real-world graph structures. For instance, the 2019 Facebook social network comprises 2.7 billion users [12], exemplifying the immense scale of real-world graphs. **Managing such large-scale graphs through full-batch GNN training, where all nodes are processed together to update parameters, is infeasible.** Nevertheless, previous mini-batch GNN training methods often suffer from huge information loss, since the connections and cross-community nodes are removed when partitioning the large-scale graphs into subgraphs.

To solve the issues, researchers have proposed sampling-based methods to train GNNs based on a mini-batch of nodes, which only aggregate the embeddings of a sampled subset of neighbors of each node in the mini-batch. VR-GCN [2] incorporates stochastic variance reduction strategies to enhance sampling efficiency during training. Cluster-GCN [4] adopts a subgraph-centric approach by confining neighbor sampling to structurally coherent clusters pre-identified through graph partitioning algorithms prior to model training. Nevertheless, existing approaches continue to struggle with preserving topological fidelity when simplifying neural architectures or optimizing training workflows. This limitation becomes particularly pronounced when scaling to deep graph neural networks with high-dimensional feature spaces, where architectural simplifications may exacerbate the loss of critical structural information.

In this paper, we propose a novel subgraph-sampling-based mini-batch training method called CGST to efficiently train GNNs without losing too much information. Rather than building a GNN on the full training graph and then sampling across the layers, we first sample the training graph and partition it into several chordal subgraphs with a balanced size. By doing so, CGST can significantly reduce the CPU memory required for GNN training, making it more scalable to larger graph datasets. Besides, CGST also applies a random aggregation technique among subgraphs to alleviate the influence of partitioning the whole graph. By building a complete GNN model on the subgraphs, CGST aims to maintain the performance characteristics of the full-batch training, avoiding the accuracy degradation seen in some prior sampling-based approaches. This allows us to capture the inherent node dependencies within the sampled subgraphs while avoiding the need to store the entire computation graph in GPU memory during back-propagation.

Our main contributions can be summarized below:

- **Balanced Chordal Subgraph Partition Module.** To solve the first challenge that previous graph partition methods make it difficult to form tight clusters, we build a graph partition module to split the original graph into subgraphs with a balanced size.
- **Random Aggregation Module.** To solve the second challenge that graph clustering algorithms tend to remove edges and cross-community nodes from the original datasets, we propose a random aggregation module to incorporate between-cluster links and reduce variance across batches.

– **Evaluation on three large-scale datasets.** Under extensive experiments on three large-scale datasets, we show that CGST provides consistent boosts in the performance of node classification tasks over large-scale graph datasets.

2 Related Works

2.1 Layer-Wise-Sampling Methods

Stochastic layer-wise sampling strategies have emerged as essential components for efficient batch-based optimization. FastGCN [3] adopts a decoupled layer sampling paradigm, executing independent node selection at each network depth rather than tracking cross-layer dependencies. The framework employs importance sampling with probability-weighted node selection to minimize estimator variance, achieving uniform sampling density across all neural architecture tiers. Besides, LADIES [17] is a layer-dependent importance sampling algorithm for efficient training of deep graph convolutional network models. In particular, LADIES aims to address the challenges of high training overhead and sparse sampling.

2.2 Subgraph-Wise-Sampling Methods

Cluster-GCN [4] proposes graph clustering-based minibatch training. During pre-processing, the training graph is partitioned into densely connected clusters. During training, clusters are randomly selected to form mini-batches, and intra-cluster edge connections remain unchanged.

2.3 Decoupling-Based Methods

APPNP [9] integrates PageRank mechanisms with graph convolutional networks to achieve effective multi-hop propagation through iterative neighborhood aggregation. The SIGN framework [7] employs a similar paradigm by implementing parallelized linear transformations that perform multi-scale neighborhood aggregations in a single network layer. This architecture subsequently feeds the concatenated propagation outputs into a standard multi-layer perceptron for downstream node-level predictions.

3 Premilinary

3.1 Problem Definition

Problem Definition 1 ***Node Classification.***

Consider a weighted graph structure $\mathcal{G}(V, E, W)$ with partial node labeling $V_l \subseteq V$, where the vertex set V (potentially enriched with additional attributes) contains n elements. The complementary subset $V_u := V \setminus V_l$ represents unannotated nodes. Here, W denotes the adjacency weight matrix while E corresponds

to the edge connections. Let the label space Y comprise m distinct classes, with $Y_l = \{y_i\}_{i=1}^{l}$ denoting the initial annotations assigned to V_l. The predictive modeling objective involves estimating complete label assignments $\widetilde{Y}$ across all vertices in V.

3.2 Background

Graph Neural Networks. A GNN learns a representation of an un-directed, attributed graph $G = (\mathcal{V}, \mathcal{E}, A)$, where each node $v \in \mathcal{V}$ has a length-f attribute x_v. Suppose A is the adjacency matrix and $\widetilde{A}$ is the normalized one (i.e., $\widetilde{A} = D^{-1}A$, and D is the diagonal degree matrix), we have:

$$\widetilde{A} = A + I, \widetilde{D}_{ii} = \sum_{j} \tilde{A}_{ij} \tag{1}$$

4 Methodology

4.1 Overview

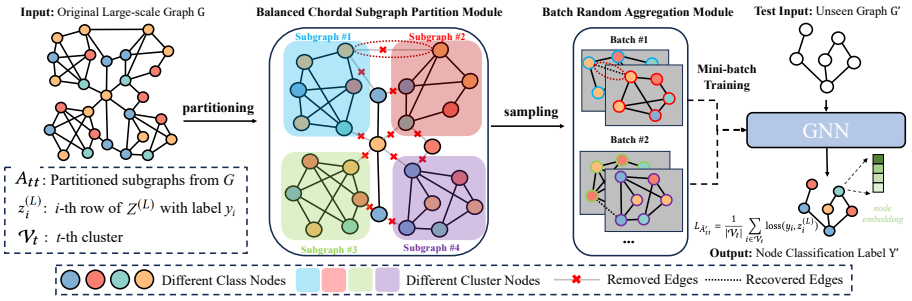


Fig. 1. The overall architecture of CGST.

Overall Architecture. In order to maintain prior information of the original graph structure while processing mini-batch training, we propose a **C**hordal **G**raph **S**ampling-based mini-batch **T**raining algorithm for GNNs on large scale graph datasets, called **CGST**. CGST includes two modules: a balanced chordal graph partition module and a batch random aggregation module. The first module extracts appropriately connected subgraphs so that little information is lost when propagating within the subgraphs. A graph partition method is applied to generate several well-partitioned chordal subgraphs, which means chordal graphs with balanced sizes. The second module solves the challenge that graph clustering algorithms tend to remove edges and cross-community nodes from the original datasets. By incorporating these two modules, CGST can achieve better performance without increasing excessive training time and training costs. The overview of CGST is shown in Figure 1.

4.2 Balanced Chordal Graph Partition

In this module, our goal is to extract appropriately connected subgraphs so that little information is lost when propagating within the subgraphs. We improve a previous graph partition method to generate several well-partitioned chordal subgraphs, which means chordal graphs with balanced sizes. In mathematical, a chordal graph can be formally defined as below:

Definition 1 ***Chordal graph****. A graph is chordal if every cycle of length at least 4 has a chord. A vertex v of G is called simplicial in G if $N(v)$ is a clique in G. The ordering $\{v_1, \ldots, v_n\}$ of the vertices of G is a perfect elimination order of G if for all i, v_i is simplicial in $G[v_1, \ldots, v_i]$. Also, a graph is chordal if it has a perfect elimination order.*

Similarly, according to [1], the well-partitioned chordal graph can be formally defined as below:

Definition 2 ***Well-partitioned chordal graph****. A graph is a well-partitioned chordal graph if and only if it has no induced subgraph isomorphic to a graph in $\mathcal{O}$. Furthermore, there is a polynomial-time algorithm that given a graph G, outputs either an induced subgraph of G isomorphic to a graph in $\mathcal{O}$, or a partition tree of each connected component which confirms that G is a well-partitioned chordal graph.*

As is proved in [1], polynomial-time decomposition of graphs into well-partitioned chordal subgraphs is computationally feasible. Structural analysis reveals that all well-partitioned chordal graphs G inherently satisfy chordality criteria: Each leaf node in the partition tree of G's components contains a simplicial vertex. Furthermore, the residual graph obtained after removing this vertex maintains the well-partitioned chordal property. This recursive characteristic enables systematic construction of perfect elimination orderings for generating such subgraphs. The complete methodology for identifying balanced chordal graph partitions is formally outlined in Algorithm 1.

4.3 Random Aggregation

In this module, in order to fetch each node i's neighbor nodes' embeddings, we need to further aggregate each neighbor node's embeddings as well. After the graph is partitioned, some original links are removed. Thus the performance could be affected. Besides, most graph clustering algorithms tend to bring similar nodes together. Hence the distribution of a cluster could be different from the original data set, leading to a biased estimation of the full gradient while performing SGD updates. In order to tackle the aforementioned challenges, we introduce a batch random aggregation mechanism proposed in [4] to integrate inter-cluster connections and reduce variability across batches. Initially, we segment the graph into clusters denoted as k clusters $C_1, \cdots, C_k$ with a relatively large value of k. When forming a batch B for a Stochastic Gradient Descent

Algorithm 1: Find balanced chordal subgraphs $G_0, \ldots, G_k$

Input: The whole graph G and a clique tree T_G of it.
Output: Balanced subgraph partitions of G.

Function *FindBalancedPartition*(T_{G_i})
if $|V(T_{G_i})| \leq \alpha \times l$ **then**
  return $\emptyset$
end if
Select an arbitrary edge e
Split T_G into two cuts T_{G_1} and T_{G_2}
$G_1 = \bigcup_{N \in V(T_{G_1})} \backslash e$, $G_2 = \bigcup_{N \in V(T_{G_2})} \backslash e$
Repeat *FindBalancedPartition* $(T_{G_1}) \cup$ *FindBalancedPartition*(T_{G_2})
End function

(SGD) update, instead of selecting a single cluster, we randomly pick n clusters, denoted as $c_1, \ldots, c_n$ and include their nodes $V_{c_1} \cup \cdots \cup V_{c_n}$ into the batch. Moreover, the connections between the chosen clusters are reintroduced. This module ensures the reintegration of inter-cluster links and reduces batch-to-batch variance through diverse cluster combinations.

5 Experiments

5.1 Experiment Setup

Datasets We evaluate our model on three large-scale graph datasets through the node classification task. Some basic information of these three datasets is demonstrated as below:

- **Twibot-22** [5]. Twibot-22 is a comprehensive graph-based Twitter bot detection benchmark that presents the largest dataset to date, provides diversified entities and relations on the Twitter network, and has considerably better annotation quality than existing datasets.
- **Hyperlink Graph** [11]. Hyperlink graphs have been extracted from the 2012 and 2014 versions of the Common Crawl web corpera. We use the 2014 graph, which covers 1.7 billion web pages connected by 64 billion hyperlinks.
- **MalNet** [8]. MalNet is a large public graph database, representing a large-scale ontology of software function call graphs. MalNet contains over 1.2 million graphs, averaging over 17k nodes and 39k edges per graph, across a hierarchy of 47 types and 696 families.

Baseline Methods We compare our model with several baseline methods for large-scale graph datasets, including sampling-based and decoupling-based methods. We select six baselines [4,15,6,17,13,7] to evaluate the performance of our model on the node classification task.

Experiment Settings We implement our proposed method CGST in PyTorch. For all the methods, we use the Adam optimizer with a learning rate as 0.01, a dropout rate as 20%, weight decay as zero. All the experiments are conducted on four machines with two NVIDIA 3090 GPUs and 128 GB memory on Ubuntu 20.04. Codes are available at `https://github.com/PoXZ0803/CGST`.

5.2 Experiment Results

Table 1. Performance and efficiency comparison between CGST and other baseline methods on three large-scale graph datasets. Four metrics in terms of F1 score(%), accuracy(%), memory usage(MB) and training time(hour) are evaluated. All experiments are repeated three times. For F1 score and accuracy, the mean and standard deviation (±) are reported. For memory usage and training time, the average scores are reported. The best results are in bold and the second best results are underlined.

Method		Twibot				Hyperlink Graph				MalNet			
		F1 Score	**Acc**	**Mem Usage**	**Training Time**	**F1 Score**	**Acc**	**Mem Usage**	**Training Time**	**F1 Score**	**Acc**	**Mem Usage**	**Training Time**
Subgraph-wise Sampling	ClusterGCN	58.62 ±1.25	90.30 ±0.89	2462	2.1	35.27 ±0.49	49.19 ±0.32	6591	3.7	41.45 ±2.60	45.28 ±1.87	6815	23.1
	GraphSAINT	66.76 ±1.46	93.05 ±0.93	2026	2.6	37.46 ±0.14	53.64 ±0.10	**4605**	**2.1**	50.47 ±0.60	53.00 ±0.87	8562	18.5
Layer-wise Sampling	GnnAutoScale	53.76 ±0.57	87.51 ±0.41	**1010**	3.5	OOM	OOM	OOM	OOM	20.90 ±0.36	26.79 ±0.32	**4406**	15.4
	LADIES	56.99 ±0.65	89.90 ±0.39	3858	3.6	41.02 ±0.23	55.86 ±0.18	5622	3.8	38.57 ±0.20	46.08 ±0.31	8830	12.6
Decoupling	SGC	51.01 ±0.80	80.64 ±1.00	4320	**0.3**	38.23 ±0.62	49.47 ±0.35	6451	2.5	30.90 0.12	36.81 0.29	8905	**9.4**
	SIGN	53.07 ±1.03	83.62 ±0.82	4463	0.5	OOM	OOM	OOM	OOM	34.13 ±0.40	37.76 ±0.28	10979	10.4
	SAGN	52.58 ±0.85	84.46 ±0.74	4719	**0.3**	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
Ours	**CGST**	**66.91 ±1.18**	**93.47 ±0.77**	1804	2.5	**45.31 ±0.35**	**57.18 ±0.39**	10499	4.3	**53.20 ±0.12**	**54.29 ±0.07**	10370	26.3

A comparison between our methods and other baseline methods on three large-scale graph datasets is shown in Table 1.

Performance. First we compare CGST with other methods in terms of F1 score and accuracy. As is shown in Table 1, our proposed CGST can achieve the highest F1 score and accuracy score among all the methods, without increasing excessive memory usage and training time. One surprising thing is that the subgraph sampling-based methods such as ClusterGCN and GraphSAINT can achieve higher accuracy than the decoupling-based methods. This is probably because the graph data is incomplete and noisy, and the stochastic nature of the sampling method can bring in regularization for training a more robust graph neural network with better generalization accuracy.

Efficiency. For training large-scale GNNs, besides performance, memory usage needed for training and training time are important and will directly restrict the scalability. The memory usage includes the memory needed for training the GCN for many epochs. In Table 1, it can be seen that when maintaining more information to boost performance, CGST's memory usage and training time do not increase a lot. The reason is that the essence of preserving information in CGST is to retain the structural information of the original graph when generating a batch, which will not bring too much memory cost. Also, a more reasonable batch division proposed in Section 4.2 can also save CGST from excessive training time.

5.3 Ablation Study

Table 2. Performance comparison between CGST and its two variants. Four metrics including F1 score, accuracy, memory usage, and training time are reported. All experiments are repeated three times. For memory usage and training time, the average scores are reported.

Method	Twibot				Hyperlink Graph				MalNet			
	F1 Score	**Acc**	**Mem Usage**	**Training Time**	**F1 Score**	**Acc**	**Mem Usage**	**Training Time**	**F1 Score**	**Acc**	**Mem Usage**	**Training Time**
CGST	**68.04 ±0.61**	**93.89 ±0.64**	2306	3.7	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
w/o CGPM												
CGST	62.92 ±3.53	87.34 ±4.88	**1633**	4.8	34.00 ±0.73	51.08 ±0.62	**7551**	5.3	51.46 ±0.21	47.30 ±0.10	**8608**	30.1
w/o BRAM												
CGST	66.91 ±1.18	93.47 ±0.77	1804	**2.5**	**45.31 ±0.35**	**57.18 ±0.39**	10499	**3.3**	**53.20 ±0.12**	**54.29 ±0.07**	10370	**26.3**

As discussed in Section 4, CGST includes two novel modules: a balanced chordal graph partition module (denoted as CGPM) and a batch random aggregation module (denoted as BRAM). We perform an ablation study to examine the effect of these two modules. To evaluate, we consider two variants of CGST:

1. CGST without chordal graph partition module (abbreviated as CGST w/o CGPM).
2. CGST without batch random aggregation module (abbreviated as w/o BRAM).

Ablation study results on three large-scale datasets are shown in Table 2, from which we could observe that these two modules improve CGST from two different perspectives. Table 2 reveals insights into the individual contributions of the CGPM and BRAM modules. The absence of the CGPM module leads to a noticeable decline in performance metrics, particularly in scenarios where graph structure is pivotal for model efficacy. Although the performance CGST w/o

CGPM on a median scale dataset (e.g., Twibot) is slightly better than that of CGST due to the removal of the constraint on subgraph size, the unbalanced number of subgraphs greatly limits its scalability, making it unable to support node classification tasks on larger datasets like Hyperlink Graph and MalNet.

5.4 Parameter Sensitivity Analysis

There exists one important hyper-parameters that should be conducted, which is the ratio of the maximum number of nodes in a subgraph to the whole graph (denoted as α). We test the sensitivity of α using the regular experiment setting on Twibot and Hyperlink Graph dataset. We vary the α from $\{0.01, 0.02, 0.05\}$, $\{1e^{-5}, 2e^{-5}, 5e^{-5}\}$ on the Twibot dataset and Hyperlink graph dataset respectively. The corresponding F1 score and training time are respectively shown in Figure 2 with blue lines. The blue shaded area in each line chart indicates the error range of the corresponding standard deviation. From the line chart in Figure 2, without affecting the scalability of CGST, α is not sensitive over these two datasets. In summary, the selection of α is a trade-off between performance and efficiency. A larger α allows CGST to retain more original graph structure information during mini-batch training and then boost performance, but it will also increase training time due to batch imbalance.

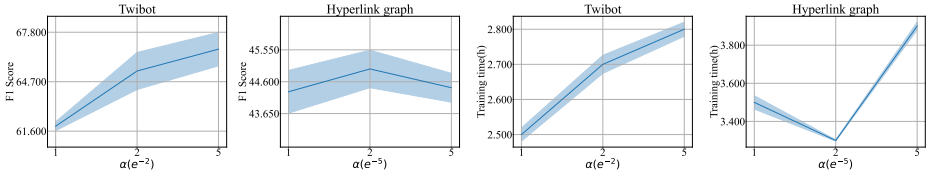


Fig. 2. Parameter Sensitivity analysis of α over Twibot dataset and Hyperlink graph dataset. The corresponding F1 scores and training time are respectively shown using line charts. The blue shaded area in each line chart indicates the error range of the corresponding standard deviation.

6 Conclusion

We propose a new algorithm namely CGST for scaling GNNs to large-scale graph datasets. CGST includes two modules: a balanced chordal graph partition module and a batch random aggregation module. The first module extracts appropriately connected subgraphs so that little information is lost when propagating within the subgraphs. A graph partition method is applied to generate several well-partitioned chordal subgraphs, which means chordal graphs with balanced sizes. The second module solves the challenge that graph clustering algorithms tend to remove edges and cross-community nodes from the original datasets. Finally, under extensive experiments on four real-world datasets, we show that CGST provides consistent boosts in the performance of node classification tasks over large-scale graph datasets.

References

1. Ahn, J., Jaffke, L., Kwon, O.j., Lima, P.T.: Well-partitioned chordal graphs. Discrete Mathematics **345**(10), 112985 (2022)
2. Chen, J., Zhu, J., Song, L.: Stochastic training of graph convolutional networks with variance reduction. arXiv preprint arXiv:1710.10568 (2017)
3. Chen, J., Ma, T., Xiao, C.: Fastgcn: Fast learning with graph convolutional networks via importance sampling. In: International Conference on Learning Representations (ICLR) (2018)
4. Chiang, W., Liu, X., Si, S., Li, Y., Bengio, S., Hsieh, C.: Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. CoRR **abs/1905.07953** (2019), http://arxiv.org/abs/1905.07953
5. Feng, S., Tan, Z., Wan, H., Wang, N., Chen, Z., Zhang, B., Zheng, Q., Zhang, W., Lei, Z., Yang, S., Feng, X., Zhang, Q., Wang, H., Liu, Y., Bai, Y., Wang, H., Cai, Z., Wang, Y., Zheng, L., Ma, Z., Li, J., Luo, M.: Twibot-22: Towards graph-based twitter bot detection (2022). https://doi.org/10.48550/ARXIV.2206.04564, https://arxiv.org/abs/2206.04564
6. Fey, M., Lenssen, J.E., Weichert, F., Leskovec, J.: Gnnautoscale: Scalable and expressive graph neural networks via historical embeddings. In: International conference on machine learning. pp. 3294–3304. PMLR (2021)
7. Frasca, F., Rossi, E., Eynard, D., Chamberlain, B., Bronstein, M., Monti, F.: Sign: Scalable inception graph neural networks (2020)
8. Freitas, S., Dong, Y., Neil, J., Chau, D.H.: A large-scale database for graph representation learning. arXiv preprint arXiv:2011.07682 (2020)
9. Gasteiger, J., Bojchevski, A., Günnemann, S.: Predict then propagate: Graph neural networks meet personalized pagerank. arXiv preprint arXiv:1810.05997 (2018)
10. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: ICLR (2017)
11. Lehmberg, O., Meusel, R., Bizer, C.: Graph structure in the web: aggregated by pay-level domain. In: Proceedings of the 2014 ACM conference on Web science. pp. 119–128 (2014)
12. Leskovec, J., Mcauley, J.: Learning to discover social circles in ego networks. Advances in neural information processing systems **25** (2012)
13. Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K.: Simplifying graph convolutional networks. In: International conference on machine learning. pp. 6861–6871. PMLR (2019)
14. Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., Leskovec, J.: Graph convolutional neural networks for web-scale recommender systems. In: KDD (2018)
15. Zeng, H., Zhou, H., Srivastava, A., Kannan, R., Prasanna, V.: Graphsaint: Graph sampling based inductive learning method. arXiv preprint arXiv:1907.04931 (2019)
16. Zhang, M., Chen, Y.: Link prediction based on graph neural networks. In: NIPS (2018)
17. Zou, D., Hu, Z., Wang, Y., Jiang, S., Sun, Y., Gu, Q.: Layer-dependent importance sampling for training deep and large graph convolutional networks. Advances in neural information processing systems **32** (2019)